# Skill-Based Assessment — GitHub Copilot

## AI Augmented Software Engineer — Practitioner Level

---

### Assessment Format

**Type:** Integrated Case Study **Duration:** 120 minutes **Tool:** GitHub Copilot (with Copilot Chat enabled) **Technology:** Your choice — any language and framework you are proficient in **Submission:** Zip file containing all required documents

---

## Case Study: TaskBridge — Notification & Audit Service

### Background

You are a software engineer at **TaskBridge**, a B2B SaaS company building a project collaboration platform for distributed engineering teams. The platform exposes a set of microservices that teams integrate into their internal tooling. The product team has approved a new **Notification & Audit Service** for the upcoming sprint. This service will sit alongside the existing **Project Service** and handle real-time notifications when project milestones are updated, as well as maintain an immutable audit log of all state changes for compliance purposes.

You've been assigned this feature. Your tech lead has given you the following brief, a piece of existing code to work with, and a set of expectations for delivery.

---

### Part 1 — Tech Lead's Brief

*"Here's what we need this sprint. The product team wants a Notification & Audit Service that sits between our Project Service and the clients. When a project milestone changes — created, updated, or closed — the service needs to: (a) emit a notification to the relevant team members, and (b) write an immutable audit entry capturing who changed what and when. Clients should be able to query audit history for a given project, filtered by date range or event type.*

*Before you build anything new, I need you to deal with some inherited code. A contractor used Copilot to build the Project Service last sprint. It was committed quickly and has never been reviewed. It's sitting in src/projects/. I want a proper review — architectural, security, the lot — before we wire the new service on top of it.*

*One more thing — midway through the sprint, the product team may drop a change request on us. If that happens, I need a quick impact analysis before anyone touches code. Document it properly.*

*Use GitHub Copilot throughout. Set up the custom instructions file if it doesn't exist. I want clean multi-service design, documented decisions, good tests, and a PR I can review by end of sprint."*

---

### Part 2 — Getting Started

You need to create a project structure as given below:

```
taskbridge-api/
├── .github/
│   └── copilot-instructions.md
├── src/
│   ├── projects/
│   │   ├── [model file]          # AI-generated, unreviewed
│   │   └── [service file]        # AI-generated, unreviewed
│   └── notifications/            # Empty — your new service
├── tests/                        # Empty
├── README.md                     # Needs your tech stack declaration
└── [dependency file]
```

**IMPORTANT: Ensure you instructed GitHub Copilot to save all the prompts in a separate file for reference.**

**Scenario:** The Project Service (`src/projects/`) was generated by a contractor using a rushed, low-effort Copilot prompt. **It has not been reviewed.** Expect issues — the code was committed without scrutiny.

**Important Note:** The Project Service files are not provided. You will **generate them yourself** by running the following low-effort prompt in GitHub Copilot, **as-is, without modification**:

> *"Generate a Project model and a Project service with create, update status, get by team, and delete functions. Use a database."*

Save the output files immediately (unreviewed AI generation). **Do not modify anything.** This simulates inheriting unreviewed AI-generated code from a contractor.

---

### Part 3 — Your Deliverables

Deliver the following as a single, cohesive body of work — not isolated exercises. Everything should fit together as a real sprint delivery.

---

#### A. Project Standards Setup

Before writing any feature code, set up `.github/copilot-instructions.md` for the project. This should define your technology stack, architecture conventions for a multi-service layout, coding standards, security rules relevant to a multi-tenant B2B SaaS context (authentication,

authorisation, data exposure), and testing expectations — comprehensive enough that any developer on the team using Copilot will get consistent, standards-compliant output.

Additionally, before writing any implementation code for the Notification & Audit Service, produce a **SPEC.md** (1–2 pages) that translates the product requirements into a clear technical specification. Include: data models with field types, API contracts (request/response shapes), integration points with the Project Service, and constraints (immutability, authorisation, validation rules). Note in the document where Copilot helped draft or refine the spec, and where you had to apply your own judgment to correct or complete it.

---

**B. Project Service — Review & Remediation**

Review the AI-generated Project Service you created earlier.

Produce:

- **REVIEW.md** — A structured code review documenting every issue you found:
  - What the issue is, where it is, its severity, and its impact (especially in a multi-tenant B2B SaaS context)
  - How you detected it — your review process, including where Copilot helped and where your own judgment was needed
  - The fix you applied or recommend
  - A section at the end: **"Architectural & Security Issues Copilot Introduced That Required Human Judgment"** — what did the AI get wrong that only a developer would catch, and why are these issues particularly risky when the service will be depended on by other services?
- **Remediated Project Service code** — Rewrite the Project Service to production standards:
  - Proper layered architecture: model → repository → service → controller/route
  - ORM-based data access (no raw database driver)
  - Input validation, typed request/response contracts, specific error handling, structured logging
  - Multi-tenant isolation: users may only access projects belonging to their organisation
  - Documentation on all public methods/functions

---

**C. Notification & Audit Service — New Build**

Build the Notification & Audit Service on top of the remediated Project Service.

- **Audit Log Model:** An audit entry capturing: event type, entity type, entity ID, actor (user ID + organisation), previous state snapshot, new state snapshot, and timestamp. Audit entries

must be immutable once written — no update or delete operations permitted.

- **Notification Model:** A notification record capturing: recipient user ID, event type, project ID, message, read status, and created timestamp.
- **Core Service Logic** that:
  - On any Project milestone state change (create / update-status / delete), writes an audit entry with before/after state and creates notification records for all relevant team members
  - Enforces immutability of audit entries at the service layer
  - Exposes audit history queryable by project ID with optional filters: date range and event type
- **API Endpoints:**
  - `POST /audit` — Internal endpoint: record an audit event (called by Project Service)
  - `GET /audit/:projectId` — Get audit history for a project (query params: `from`, `to`, `eventType`)
  - `GET /notifications/:userId` — Get all unread notifications for a user
  - `PATCH /notifications/:id/read` — Mark a notification as read
- **Scope Change — Impact Analysis:** Midway through the sprint, the product team issues the following change request: *"Add a new milestone event type: `MILESTONE_REOPENED`. This should trigger audit logging and notifications. Audit entries must now also capture the actor's IP address."* Before touching any code, document your impact analysis in **IMPACT_ANALYSIS.md**:
  - Every file, module, and data model affected, and the nature of each change (additive, breaking, migration required)
  - Security and compliance risks introduced by capturing IP addresses (privacy, data retention, logging exposure)
  - Recommended implementation approach and sequencing
  - A section: **"How Copilot Assisted This Analysis"** — what you prompted, what it produced, and where you had to validate or override its output
- **Tests** — Minimum 6 test cases covering:
  - Equal notification dispatch to all team members on a project state change
  - Audit entry is created correctly when a project milestone is updated
  - Audit entry cannot be deleted or overwritten (immutability enforcement)
  - Audit history query returns correct results filtered by date range
  - Audit history query filtered by event type returns only matching entries
  - Unauthorised user cannot access another organisation's audit log

---

**D. Prompt Engineering Documentation**

Create **PROMPTS.md** documenting how you used GitHub Copilot to build the Notification & Audit Service and the SPEC.md:

- The prompt chain you used, in the order you executed them
- For each prompt: the exact text, which Copilot feature you chose, which prompting technique you applied (specificity, decomposition, few-shot, constraint, role-based, iterative refinement), and a brief rationale for your approach
- Your chain must demonstrate use of **at least 2 different Copilot features** and **at least 3 different prompting techniques**
- A closing section: **"Post-Generation Corrections"** — every change you made to Copilot's output, what was wrong, and how you fixed it

---

**E. Collaboration Artifacts**

Prepare this work as if it's going to a real pull request:

- **Commit history:** Minimum 5 logical commits using Conventional Commits format, each with a descriptive body. Your commit history should tell the story of your work: standards setup → project service review/fix → spec + service build → tests → documentation and impact analysis.
- **PR_DESCRIPTION.md** with:
  - Summary of what was built and why
  - **AI Tool Disclosure:** Which Copilot features you used, where you accepted AI output vs overrode it, and your estimate of AI-generated vs hand-written code percentage
  - How the two services integrate and any inter-service contracts established
  - Testing coverage and any known gaps
  - At least 1 genuine risk or trade-off in your multi-service design
  - Self-review checklist you ran before submitting
  - **3 peer review comments** (in PR_DESCRIPTION.md under "Peer Review Simulation") — written as if reviewing a teammate's version of the Notification & Audit feature. Each comment must be specific (code location), actionable (what to change), and constructive (why). At least 1 must address something AI tools would typically miss.

---

**F. Tool Strategy Reflection**

Create **TOOL_STRATEGY.md** documenting:

- **Feature Usage Log:** How you used Copilot across this case study — minimum 6 entries covering at least 4 different Copilot features. For each: what you used, why that feature (not another), and what happened.
- **Scenario Responses:** For each scenario below, name the **specific Copilot feature** you'd use and explain why in 2–3 sentences:
  - Understanding a complex 600-line legacy service in an unfamiliar codebase before wiring a new service to it
  - Generating consistent, standards-compliant request-validation middleware across 10 existing route handlers
  - Quickly verifying whether a JWT verification implementation correctly handles token expiry and signature tampering
  - Enforcing that all commits to main pass linting and test coverage thresholds automatically, with no human intervention
  - Reviewing a contractor's AI-generated service module for security vulnerabilities before it reaches staging
  - Ensuring Copilot follows multi-tenant data isolation rules consistently across all developers and sessions
- **Limitations Encountered:** 3 real situations from this case study where Copilot produced incorrect, incomplete, or inappropriate output — what you prompted, what went wrong, how you detected it, how you fixed it, and what you'd do differently. *(Claiming zero limitations = 0 points for this section.)*

---

**Part 4 — Architecture Documentation**

Create **ARCHITECTURE.md** (10–15 lines) covering:

- How the Project Service and Notification & Audit Service relate to each other, including the integration contract between them
- Your layered architecture and data flow from an inbound API request through to audit and notification persistence
- Why this architecture is appropriate for a multi-tenant B2B SaaS application
- Key design decisions you made and the trade-offs you considered

---

## Submission Checklist

Your zip file must contain:

| **Deliverable** Please refer detailed associate guide to follow all instructions for submissions. |
| --- |
| README.md with technology stack declared |
| .github/copilot-instructions.md |
| SPEC.md — Technical specification for Notification & Audit Service |
| REVIEW.md (with "Architectural & Security Issues" section) |
| Remediated Project Service (model, repository, service, controller) |
| Notification & Audit Service (model, service, controller) |
| Test file(s) with ⩾ 6 test cases |
| IMPACT_ANALYSIS.md (scope change + Copilot-assisted analysis) |
| PROMPTS.md (with "Post-Generation Corrections" section) |
| PR_DESCRIPTION.md (with AI Disclosure + Peer Review Simulation) |
| TOOL_STRATEGY.md (Usage Log + Scenarios + Limitations) |
| ARCHITECTURE.md |